G101 BASES DE DATOS II

TAREA. CONSTRUYENDO UNA BASE DE DATOS NOSQL. VALOR 10%

SANTIAGO PADIERNA SUAREZ

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
FACULTAD DE INGENIERÍA

ORALIA CORTES GRAJALES

TECNOLOGÍA EN DESARROLLO DE SOFTWARE

2026-2

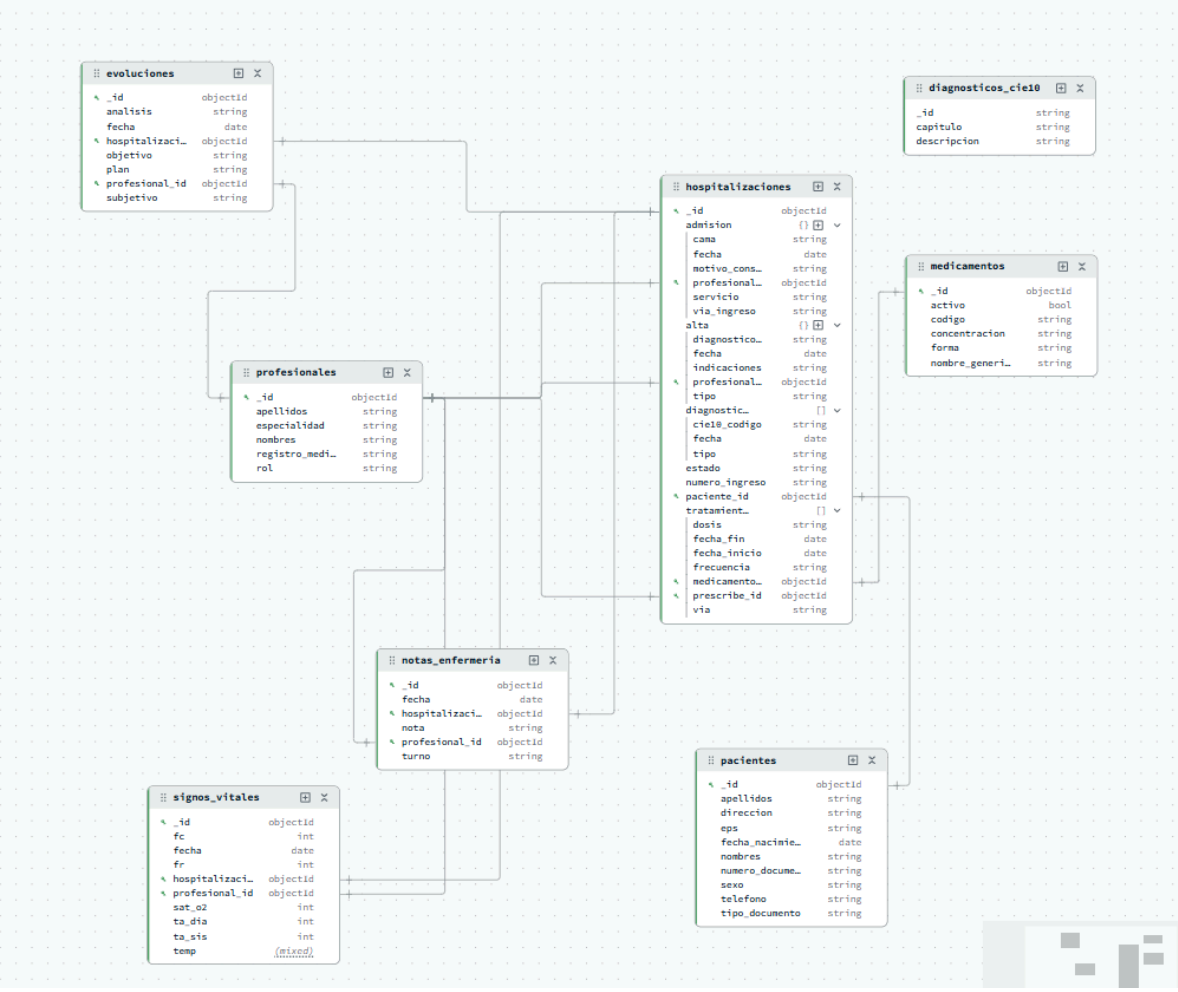
0. Anotaciones

Video:
https://drive.google.com/file/d/1eEk1aKHz8lRbbWovJGwYXY_Lwq41mgMq/view?usp=drive_link

Github: https://github.com/soypadierna/hce-hospitalizacion-mongodb

1. Esquema del modelo

Modelo documental híbrido para el proceso de hospitalización de una Historia Clínica Electrónica (piloto Gobernación de Antioquia). Base de datos: hce_hospitalizacion. Todos los datos son ficticios.



pacientes (datos identificables, aislados)

Campo	Tipo	Oblig.	Modo
tipo_documento, numero_documento	enum / string	Sí	Identificador único (índice único)
nombres, apellidos	string	Sí	Solo en esta colección
fecha_nacimiento	date	Sí	
sexo	enum (M, F, O)	Sí	
telefono, direccion, eps	string	No	Contacto

hospitalizaciones (datos clínicos)

Campo	Tipo	Oblig.	Modo
paciente_id	ObjectId	Sí	Referenciado
numero_ingreso	string	Sí	Índice único
estado	enum (activa, alta)	Sí	
admision	objeto	Sí	Embebido: tamaño acotado, se lee siempre junto
diagnosticos[]	array (1 a 20)	Sí	Embebido; cie10_codigo referenciado
tratamientos[]	array (máx. 100)	No	Embebido; medicamento_id y prescribe_id referenciados
alta	objeto	No	Embebido: único y acotado

signos_vitales, notas_enfermeria, evoluciones (crecimiento sin tope, referenciadas)

Campo	Tipo	Oblig.	Modo
hospitalizacion_id	ObjectId	Sí	Referencia al padre
fecha	date	Sí	
profesional_id	ObjectId	Sí	Referencia al catálogo
fc, fr, temp, sat_o2, ta_sis, ta_dia	number	Sí (signos_vitales)	

nota, turno	string / enum	Sí (notas_enfermeria)	
subjetivo, objetivo, analisis, plan	string	Sí (evoluciones)	Formato SOAP

Catálogos referenciados

Colección	Campos	Clave
medicamentos	codigo, nombre_generico, forma, concentracion, activo	codigo único
profesionales	registro_medico, nombres, apellidos, rol, especialidad	registro_medico único
diagnosticos_cie10	descripcion, capitulo	_id = código CIE-10

2. Creacion de BD y Carga de datos

Script:
https://drive.google.com/file/d/1neJhyOVrhUYKZLq-2titZHrO0NBOxTA_/view?usp=drive_link

```
// hce_seed.js
(function () {
  db = db.getSiblingDB("hce_hospitalizacion");
  const F = s => ISODate(s + ":00Z");
  const oid = { bsonType: "objectId" }, str = { bsonType: "string" }, dt = { bsonType: "date" }, num = { bsonType: "number" };
  // Borrado previo de colecciones para poder re-ejecutar
  ["signos_vitales", "notas_enfermeria", "evoluciones", "hospitalizaciones", "medicamentos", "profesionales", "diagnosticos_cie10", "pacientes"].forEach(c => db.getCollection(c).drop());
  // Colecciones con validadores $jsonSchema e índices
  db.createCollection("pacientes", { validator: { $jsonSchema: {
    bsonType: "object",
    required: ["tipo_documento", "numero_documento", "nombres", "apellidos", "fecha_nacimiento", "sexo"],
    properties: {
      tipo_documento: { enum: ["CC", "TI", "CE", "PA", "RC"] }, numero_documento: str,
      nombres: str, apellidos: str, fecha_nacimiento: dt, sexo: { enum: ["M", "F", "O"] },
      telefono: str, direccion: str, eps: str } } } });
  db.pacientes.createIndex({ tipo_documento: 1, numero_documento: 1 }, { unique: true });
  db.createCollection("profesionales", { validator: { $jsonSchema: {
    bsonType: "object", required: ["registro_medico", "nombres", "apellidos", "rol"],
    properties: { registro_medico: str, nombres: str, apellidos: str,
```

```
    rol: { enum: ["medico", "enfermero", "especialista"] }, especialidad: str } } } });
  db.profesionales.createIndex({ registro_medico: 1 }, { unique: true });
  db.createCollection("medicamentos", { validator: { $jsonSchema: {
    bsonType: "object", required: ["codigo", "nombre_generico", "forma", "concentracion", "activo"],
    properties: { codigo: str, nombre_generico: str, forma: str, concentracion: str, activo: { bsonType:
"bool" } } } } });
  db.medicamentos.createIndex({ codigo: 1 }, { unique: true });
  db.medicamentos.createIndex({ nombre_generico: "text" });
  db.createCollection("diagnosticos_cie10", { validator: { $jsonSchema: {
    bsonType: "object", required: ["_id", "descripcion"],
    properties: { _id: str, descripcion: str, capitulo: str } } } });
  db.diagnosticos_cie10.createIndex({ descripcion: "text" });
  db.createCollection("hospitalizaciones", { validator: { $jsonSchema: {
    bsonType: "object", required: ["paciente_id", "numero_ingreso", "estado", "admision",
"diagnosticos"],
    properties: {
      paciente_id: oid, numero_ingreso: str, estado: { enum: ["activa", "alta"] },
      admision: { bsonType: "object",
        required: ["fecha", "via_ingreso", "servicio", "cama", "motivo_consulta", "profesional_id"],
        properties: { fecha: dt, via_ingreso: { enum: ["urgencias", "consulta_externa", "remision"] },
          servicio: str, cama: str, motivo_consulta: str, profesional_id: oid } },
      diagnosticos: { bsonType: "array", minItems: 1, maxItems: 20, items: { bsonType: "object",
        required: ["cie10_codigo", "tipo", "fecha"],
        properties: { cie10_codigo: str, tipo: { enum: ["ingreso", "principal", "secundario", "egreso"] },
fecha: dt } } },
      tratamientos: { bsonType: "array", maxItems: 100, items: { bsonType: "object",
        required: ["medicamento_id", "dosis", "via", "frecuencia", "fecha_inicio", "prescribe_id"],
        properties: { medicamento_id: oid, dosis: str, via: { enum: ["oral", "IV", "IM", "SC", "topica",
"inhalada"] },
          frecuencia: str, fecha_inicio: dt, fecha_fin: dt, prescribe_id: oid } } },
      alta: { bsonType: "object", required: ["fecha", "tipo", "diagnostico_egreso", "profesional_id"],
        properties: { fecha: dt, tipo: { enum: ["mejoria", "remision", "voluntaria", "fallecimiento"] },
          diagnostico_egreso: str, indicaciones: str, profesional_id: oid } } } } });
  db.hospitalizaciones.createIndex({ paciente_id: 1, "admision.fecha": -1 });
  db.hospitalizaciones.createIndex({ numero_ingreso: 1 }, { unique: true });
  db.hospitalizaciones.createIndex({ estado: 1, "admision.servicio": 1 });
  db.hospitalizaciones.createIndex({ "diagnosticos.cie10_codigo": 1 });
  const hijo = (extra, req) => ({ validator: { $jsonSchema: { bsonType: "object",
    required: ["hospitalizacion_id", "fecha", "profesional_id", ...req],
    properties: { hospitalizacion_id: oid, fecha: dt, profesional_id: oid, ...extra } } } });
  db.createCollection("signos_vitales", hijo({ fc: num, fr: num, temp: num, sat_o2: num, ta_sis: num,
ta_dia: num }, ["fc", "fr", "temp", "sat_o2", "ta_sis", "ta_dia"]));
  db.createCollection("notas_enfermeria", hijo({ nota: str, turno: { enum: ["manana", "tarde", "noche"]
} }, ["nota", "turno"]));
  db.createCollection("evoluciones", hijo({ subjetivo: str, objetivo: str, analisis: str, plan: str },
["subjetivo", "objetivo", "analisis", "plan"]));
  ["signos_vitales", "notas_enfermeria", "evoluciones"].forEach(c =>
db.getCollection(c).createIndex({ hospitalizacion_id: 1, fecha: -1 }));
  // Identificadores compartidos entre colecciones
```

```
  const pa = Array.from({ length: 6 }, () => ObjectId());
  const pr = Array.from({ length: 7 }, () => ObjectId());
  const me = Array.from({ length: 12 }, () => ObjectId());
  const ho = Array.from({ length: 6 }, () => ObjectId());
  // Catálogos
  db.diagnosticos_cie10.insertMany([
    { _id: "J18.9", descripcion: "Neumonía, no especificada", capitulo: "X" },
    { _id: "I50.9", descripcion: "Insuficiencia cardíaca, no especificada", capitulo: "IX" },
    { _id: "E11.1", descripcion: "Diabetes mellitus tipo 2 con cetoacidosis", capitulo: "IV" },
    { _id: "N39.0", descripcion: "Infección de vías urinarias, sitio no especificado", capitulo: "XIV" },
    { _id: "J44.1", descripcion: "Enfermedad pulmonar obstructiva crónica con exacerbación aguda",
capitulo: "X" },
    { _id: "I10", descripcion: "Hipertensión esencial (primaria)", capitulo: "IX" },
    { _id: "R50.9", descripcion: "Fiebre, no especificada", capitulo: "XVIII" },
    { _id: "A09", descripcion: "Diarrea y gastroenteritis de presunto origen infeccioso", capitulo: "I" },
    { _id: "K35.8", descripcion: "Apendicitis aguda, otra y la no especificada", capitulo: "XI" },
    { _id: "I21.9", descripcion: "Infarto agudo del miocardio, sin otra especificación", capitulo: "IX" }]);
  db.profesionales.insertMany([
    { _id: pr[0], registro_medico: "RM-0001", nombres: "Laura", apellidos: "Restrepo Vélez", rol:
"medico", especialidad: "Medicina interna" },
    { _id: pr[1], registro_medico: "RM-0002", nombres: "Andrés", apellidos: "Villegas Toro", rol:
"especialista", especialidad: "Cardiología" },
    { _id: pr[2], registro_medico: "RM-0003", nombres: "Marcela", apellidos: "Ospina Duque", rol:
"especialista", especialidad: "Neumología" },
    { _id: pr[3], registro_medico: "RM-0004", nombres: "Hernán", apellidos: "Quintero Pareja", rol:
"medico", especialidad: "Urgencias" },
    { _id: pr[4], registro_medico: "RE-0005", nombres: "Carlos", apellidos: "Mejía Uribe", rol:
"enfermero" },
    { _id: pr[5], registro_medico: "RE-0006", nombres: "Diana", apellidos: "Zapata Gil", rol:
"enfermero" },
    { _id: pr[6], registro_medico: "RE-0007", nombres: "Julián", apellidos: "Cardona Ruiz", rol:
"enfermero" }]);
  db.medicamentos.insertMany([
    { _id: me[0], codigo: "MED-001", nombre_generico: "Ceftriaxona", forma: "Inyectable",
concentracion: "1 g", activo: true },
    { _id: me[1], codigo: "MED-002", nombre_generico: "Azitromicina", forma: "Tableta",
concentracion: "500 mg", activo: true },
    { _id: me[2], codigo: "MED-003", nombre_generico: "Paracetamol", forma: "Tableta",
concentracion: "500 mg", activo: true },
    { _id: me[3], codigo: "MED-004", nombre_generico: "Enoxaparina", forma: "Jeringa prellenada",
concentracion: "40 mg/0.4 mL", activo: true },
    { _id: me[4], codigo: "MED-005", nombre_generico: "Furosemida", forma: "Ampolla inyectable",
concentracion: "20 mg/2 mL", activo: true },
    { _id: me[5], codigo: "MED-006", nombre_generico: "Insulina glargina", forma: "Solución
inyectable", concentracion: "100 UI/mL", activo: true },
    { _id: me[6], codigo: "MED-007", nombre_generico: "Omeprazol", forma: "Cápsula",
concentracion: "20 mg", activo: true },
    { _id: me[7], codigo: "MED-008", nombre_generico: "Salbutamol", forma: "Inhalador",
concentracion: "100 mcg/dosis", activo: true },
```

```
  { _id: me[8], codigo: "MED-009", nombre_generico: "Metoprolol", forma: "Tableta", concentracion: "50 mg", activo: true },
  { _id: me[9], codigo: "MED-010", nombre_generico: "Ácido acetilsalicílico", forma: "Tableta", concentracion: "100 mg", activo: true },
  { _id: me[10], codigo: "MED-011", nombre_generico: "Dexametasona", forma: "Solución inyectable", concentracion: "4 mg/mL", activo: true },
  { _id: me[11], codigo: "MED-012", nombre_generico: "Insulina regular", forma: "Solución inyectable", concentracion: "100 UI/mL", activo: true }]);
 db.pacientes.insertMany([
  { _id: pa[0], tipo_documento: "CC", numero_documento: "1000000001", nombres: "Juan Camilo", apellidos: "Pérez Gómez", fecha_nacimiento: F("1985-03-12T00:00"), sexo: "M", telefono: "3000000001", direccion: "Calle 1 # 2-3, Medellín", eps: "EPS Ficticia Sur" },
  { _id: pa[1], tipo_documento: "CC", numero_documento: "1000000002", nombres: "María Fernanda", apellidos: "Gómez Ruiz", fecha_nacimiento: F("1979-07-25T00:00"), sexo: "F", telefono: "3000000002", direccion: "Carrera 10 # 20-30, Bello", eps: "EPS Ficticia Norte" },
  { _id: pa[2], tipo_documento: "CC", numero_documento: "1000000003", nombres: "Luis Alberto", apellidos: "Torres Mora", fecha_nacimiento: F("1958-11-02T00:00"), sexo: "M", telefono: "3000000003", direccion: "Calle 50 # 40-10, Envigado", eps: "EPS Ficticia Sur" },
  { _id: pa[3], tipo_documento: "CC", numero_documento: "1000000004", nombres: "Ana Sofía", apellidos: "Cardona Londoño", fecha_nacimiento: F("1992-05-19T00:00"), sexo: "F", telefono: "3000000004", direccion: "Carrera 70 # 30-15, Medellín", eps: "EPS Ficticia Centro" },
  { _id: pa[4], tipo_documento: "CC", numero_documento: "1000000005", nombres: "Pedro Pablo", apellidos: "Salazar Henao", fecha_nacimiento: F("1950-01-30T00:00"), sexo: "M", telefono: "3000000005", direccion: "Calle 30 # 55-20, Itagüí", eps: "EPS Ficticia Norte" },
  { _id: pa[5], tipo_documento: "CC", numero_documento: "1000000006", nombres: "Valentina", apellidos: "Ríos Arango", fecha_nacimiento: F("2001-09-14T00:00"), sexo: "F", telefono: "3000000006", direccion: "Carrera 43A # 10-5, Medellín", eps: "EPS Ficticia Centro" }]);
 // Funciones auxiliares para construir hospitalizaciones y registros hijos
 const ad = (f, via, serv, cama, mot, prof) => ({ fecha: F(f), via_ingreso: via, servicio: serv, cama: cama, motivo_consulta: mot, profesional_id: prof });
 const al = (f, tipo, dg, ind, prof) => ({ fecha: F(f), tipo: tipo, diagnostico_egreso: dg, indicaciones: ind, profesional_id: prof });
 const dx = rows => rows.map(r => ({ cie10_codigo: r[0], tipo: r[1], fecha: F(r[2]) }));
 const tx = rows => rows.map(r => { const o = { medicamento_id: r[0], dosis: r[1], via: r[2], frecuencia: r[3], fecha_inicio: F(r[4]), prescribe_id: r[6] }; if (r[5]) o.fecha_fin = F(r[5]); return o; });
 const hs = (id, pac, num, estado, adm, diag, trat, alta) => { const d = { _id: id, paciente_id: pac, numero_ingreso: num, estado: estado, admision: adm, diagnosticos: diag, tratamientos: trat }; if (alta) d.alta = alta; db.hospitalizaciones.insertOne(d); };
 const sv = (h, rows) => db.signos_vitales.insertMany(rows.map(r => ({ hospitalizacion_id: h, fecha: F(r[0]), profesional_id: r[1], fc: r[2], fr: r[3], temp: r[4], sat_o2: r[5], ta_sis: r[6], ta_dia: r[7] })));
 const nt = (h, rows) => db.notas_enfermeria.insertMany(rows.map(r => ({ hospitalizacion_id: h, fecha: F(r[0]), profesional_id: r[1], turno: r[2], nota: r[3] })));
 const ev = (h, rows) => db.evoluciones.insertMany(rows.map(r => ({ hospitalizacion_id: h, fecha: F(r[0]), profesional_id: r[1], subjetivo: r[2], objetivo: r[3], analisis: r[4], plan: r[5] })));
 // Hospitalización 1: neumonía (Juan Camilo), con alta
 hs(ho[0], pa[0], "HOS-2026-0001", "alta",
   ad("2026-10-01T08:30", "urgencias", "Medicina interna", "M-12", "Fiebre y tos productiva de 4 días", pr[3]),
```

```
  dx([["J18.9", "ingreso", "2026-10-01T09:00"], ["R50.9", "secundario", "2026-10-01T09:00"],
["J18.9", "principal", "2026-10-02T08:00"], ["J18.9", "egreso", "2026-10-06T10:00"]]),
  tx([[me[0], "1 g", "IV", "cada 24 h", "2026-10-01T10:00", "2026-10-06T10:00", pr[0]], [me[1], "500
mg", "oral", "cada 24 h", "2026-10-01T10:30", "2026-10-04T10:30", pr[0]], [me[2], "500 mg", "oral",
"cada 8 h si fiebre", "2026-10-01T10:00", "2026-10-03T10:00", pr[0]]]),
  al("2026-10-06T11:00", "mejoria", "J18.9", "Control ambulatorio en 7 días y signos de alarma
respiratoria", pr[0]));
 ev(ho[0], [
  ["2026-10-01T16:00", pr[0], "Tos con expectoración, fiebre y malestar general.", "T 38.4 °C, FR
22, estertores en base derecha, SatO2 93%.", "Neumonía adquirida en comunidad moderada.",
"Ceftriaxona IV, azitromicina y oxígeno si hay desaturación."],
  ["2026-10-03T08:00", pr[0], "Menos tos, sin disnea.", "Afebril 24 h, SatO2 95% en aire
ambiente.", "Respuesta clínica favorable.", "Continuar antibiótico y vigilar signos vitales."],
  ["2026-10-05T08:00", pr[0], "Asintomático, tolera vía oral.", "Afebril, SatO2 96%, murmullo
vesicular conservado.", "Neumonía en resolución.", "Programar egreso al completar el
esquema."]]);
 sv(ho[0], [
  ["2026-10-01T12:00", pr[4], 92, 22, 38.4, 93, 120, 80], ["2026-10-01T20:00", pr[5], 96, 24, 38.9,
92, 118, 76],
  ["2026-10-02T08:00", pr[4], 88, 20, 37.8, 94, 122, 78], ["2026-10-03T08:00", pr[4], 82, 18, 37.1,
95, 120, 80],
  ["2026-10-04T08:00", pr[4], 78, 18, 36.8, 96, 118, 76], ["2026-10-05T08:00", pr[5], 76, 16, 36.6,
97, 116, 74]]);
 nt(ho[0], [
  ["2026-10-01T14:00", pr[4], "tarde", "Ingresa febril, se administra primera dosis de antibiótico y se
inicia oxígeno por cánula."],
  ["2026-10-02T22:00", pr[5], "noche", "Paciente descansa, sin picos febriles, tolera dieta."],
  ["2026-10-04T09:00", pr[4], "manana", "Deambula con apoyo, saturación estable en aire
ambiente."]]);
 // Hospitalización 2: neumonía (María Fernanda), mismo diagnóstico principal, con alta
 hs(ho[1], pa[1], "HOS-2026-0002", "alta",
  ad("2026-10-03T14:10", "urgencias", "Medicina interna", "M-14", "Tos, fiebre y dificultad
respiratoria de 3 días", pr[3]),
  dx([["J18.9", "ingreso", "2026-10-03T14:40"], ["J18.9", "principal", "2026-10-04T08:00"], ["I10",
"secundario", "2026-10-04T08:00"], ["J18.9", "egreso", "2026-10-09T10:00"]]),
  tx([[me[0], "1 g", "IV", "cada 24 h", "2026-10-03T15:00", "2026-10-08T15:00", pr[2]], [me[1], "500
mg", "oral", "cada 24 h", "2026-10-03T15:30", "2026-10-06T15:30", pr[2]], [me[8], "50 mg", "oral",
"cada 12 h", "2026-10-04T09:00", "2026-10-09T09:00", pr[0]]]),
  al("2026-10-09T11:30", "mejoria", "J18.9", "Control por consulta externa en 8 días y continuar
antihipertensivo", pr[2]));
 ev(ho[1], [
  ["2026-10-04T08:00", pr[2], "Persisten tos y disnea de esfuerzo.", "FR 24, SatO2 92%, crépitos
en base izquierda.", "Neumonía adquirida en comunidad en paciente hipertensa.",
"Antibioticoterapia IV, oxígeno suplementario y continuar antihipertensivo."],
  ["2026-10-06T08:00", pr[2], "Mejoría de la disnea, tos menos frecuente.", "Afebril, SatO2 95% en
aire ambiente.", "Evolución favorable.", "Continuar ceftriaxona IV y retirar oxígeno."],
  ["2026-10-08T08:00", pr[0], "Sin disnea, buen apetito.", "Afebril, SatO2 96%, TA 124/80.",
"Neumonía resuelta clínicamente.", "Egreso programado en 24 horas."]]);
 sv(ho[1], [
```

```
    ["2026-10-03T16:00", pr[4], 100, 24, 38.6, 91, 150, 92], ["2026-10-03T22:00", pr[6], 98, 24, 38.2,
92, 146, 90],
    ["2026-10-04T08:00", pr[4], 90, 22, 37.5, 92, 142, 88], ["2026-10-05T08:00", pr[5], 84, 20, 37, 94,
134, 84],
    ["2026-10-07T08:00", pr[4], 78, 18, 36.7, 96, 126, 80], ["2026-10-08T08:00", pr[5], 76, 16, 36.6,
96, 124, 80]]);
  nt(ho[1], [
    ["2026-10-03T18:00", pr[5], "tarde", "Se inicia oxígeno por cánula a 2 L/min, tolera bien."],
    ["2026-10-05T07:30", pr[6], "manana", "Refiere tos nocturna, se eleva cabecera y se administra
medicación según orden."],
    ["2026-10-07T21:00", pr[6], "noche", "Sueño continuo, sin disnea, signos vitales estables."]]);
  // Hospitalización 3: insuficiencia cardíaca (Luis Alberto), ACTIVA sin alta
  hs(ho[2], pa[2], "HOS-2026-0003", "activa",
    ad("2026-10-22T19:20", "urgencias", "Cardiología", "C-05", "Disnea progresiva y edema de
miembros inferiores", pr[3]),
    dx([["I50.9", "ingreso", "2026-10-22T20:00"], ["I50.9", "principal", "2026-10-23T08:00"], ["I10",
"secundario", "2026-10-23T08:00"]]),
    tx([[me[4], "40 mg", "IV", "cada 12 h", "2026-10-22T21:00", null, pr[1]], [me[3], "40 mg", "SC",
"cada 24 h", "2026-10-23T09:00", null, pr[1]], [me[8], "50 mg", "oral", "cada 12 h",
"2026-10-24T09:00", null, pr[1]], [me[6], "20 mg", "oral", "cada 24 h", "2026-10-23T09:00", null,
pr[1]]]),
    null);
  ev(ho[2], [
    ["2026-10-23T08:00", pr[1], "Disnea de reposo mejor, ortopnea de dos almohadas.", "Edema
grado II, crépitos bibasales, SatO2 91% en aire ambiente.", "Insuficiencia cardíaca
descompensada.", "Diuresis IV, balance hídrico estricto y restricción de sodio."],
    ["2026-10-24T08:00", pr[1], "Menos disnea, orinó abundante durante la noche.", "Peso -2 kg,
SatO2 94%, edema grado I.", "Respuesta adecuada al diurético.", "Continuar furosemida IV e
iniciar betabloqueador."],
    ["2026-10-25T08:00", pr[0], "Sin ortopnea, duerme bien.", "Edema leve, SatO2 95%, TA 118/72.",
"Mejoría progresiva, continúa hospitalizado.", "Valorar paso a diurético oral en 24 a 48 horas."]]);
  sv(ho[2], [
    ["2026-10-22T20:00", pr[6], 104, 26, 36.8, 90, 158, 96], ["2026-10-23T08:00", pr[4], 98, 24, 36.7,
91, 150, 92],
    ["2026-10-23T20:00", pr[6], 92, 22, 36.6, 93, 142, 88], ["2026-10-24T08:00", pr[4], 88, 20, 36.6,
94, 134, 84],
    ["2026-10-24T20:00", pr[6], 84, 18, 36.5, 95, 126, 78], ["2026-10-25T08:00", pr[4], 80, 18, 36.6,
95, 118, 72]]);
  nt(ho[2], [
    ["2026-10-22T22:00", pr[6], "noche", "Ingresa con edema de miembros inferiores, se inicia
monitoreo y control de líquidos."],
    ["2026-10-23T15:00", pr[4], "tarde", "Diuresis de 1800 mL en 8 horas, tolera dieta hiposódica."],
    ["2026-10-24T10:00", pr[5], "manana", "Se pesa al paciente, disminución de 2 kg, sin disnea en
reposo."]]);
  // Hospitalización 4: infección urinaria (Ana Sofía), con alta
  hs(ho[3], pa[3], "HOS-2026-0004", "alta",
    ad("2026-10-08T10:15", "urgencias", "Medicina interna", "M-07", "Fiebre, disuria y dolor lumbar
de 2 días", pr[3]),
```

```
  dx([["N39.0", "ingreso", "2026-10-08T10:45"], ["R50.9", "secundario", "2026-10-08T10:45"],
["N39.0", "principal", "2026-10-08T16:00"], ["N39.0", "egreso", "2026-10-11T10:00"]]),
  tx([[me[0], "1 g", "IV", "cada 24 h", "2026-10-08T11:00", "2026-10-11T11:00", pr[0]], [me[2], "500
mg", "oral", "cada 8 h", "2026-10-08T11:00", "2026-10-10T11:00", pr[0]]]),
  al("2026-10-11T12:00", "mejoria", "N39.0", "Completar tratamiento oral ambulatorio y aumentar
ingesta de líquidos", pr[0]));
 ev(ho[3], [
  ["2026-10-08T16:00", pr[0], "Disuria, fiebre y dolor lumbar derecho.", "T 38.7 °C, puñopercusión
positiva derecha.", "Infección urinaria alta, pielonefritis probable.", "Ceftriaxona IV, urocultivo e
hidratación."],
  ["2026-10-09T08:00", pr[0], "Fiebre menor, dolor lumbar leve.", "T 37.6 °C, hemodinámicamente
estable.", "Respuesta inicial adecuada.", "Continuar manejo intravenoso."],
  ["2026-10-10T08:00", pr[0], "Sin fiebre, sin disuria.", "Afebril 24 h, tolera vía oral.", "Mejoría
clínica.", "Planear egreso con terapia oral."]]);
 sv(ho[3], [
  ["2026-10-08T12:00", pr[5], 102, 20, 38.7, 97, 108, 68], ["2026-10-08T20:00", pr[6], 98, 20, 38.3,
97, 106, 66],
  ["2026-10-09T08:00", pr[4], 90, 18, 37.6, 98, 110, 70], ["2026-10-09T20:00", pr[6], 84, 16, 37, 98,
112, 72],
  ["2026-10-10T08:00", pr[4], 78, 16, 36.7, 99, 114, 72], ["2026-10-10T20:00", pr[6], 76, 16, 36.6,
99, 112, 70]]);
 nt(ho[3], [
  ["2026-10-08T13:00", pr[5], "tarde", "Se toma muestra de orina para cultivo y se inicia hidratación
IV."],
  ["2026-10-09T09:00", pr[4], "manana", "Refiere dolor lumbar leve, se administra analgésico
según orden."],
  ["2026-10-10T22:00", pr[6], "noche", "Duerme sin fiebre, diuresis espontánea clara."]]);
 // Hospitalización 5: EPOC exacerbado (Pedro Pablo), con alta
 hs(ho[4], pa[4], "HOS-2026-0005", "alta",
  ad("2026-10-12T05:50", "urgencias", "Neumología", "N-03", "Disnea y sibilancias de 2 días",
pr[3]),
  dx([["J44.1", "ingreso", "2026-10-12T06:20"], ["I10", "secundario", "2026-10-12T09:00"], ["J44.1",
"principal", "2026-10-12T09:00"], ["J44.1", "egreso", "2026-10-17T10:00"]]),
  tx([[me[7], "2 inhalaciones", "inhalada", "cada 4 h", "2026-10-12T06:30", "2026-10-17T06:30",
pr[2]], [me[10], "8 mg", "IV", "cada 24 h", "2026-10-12T07:00", "2026-10-16T07:00", pr[2]], [me[1],
"500 mg", "oral", "cada 24 h", "2026-10-12T09:30", "2026-10-15T09:30", pr[2]]]),
  al("2026-10-17T11:00", "mejoria", "J44.1", "Inhaladores de mantenimiento, abstinencia de tabaco
y control en 10 días", pr[2]));
 ev(ho[4], [
  ["2026-10-12T09:00", pr[2], "Disnea marcada con sibilancias y tos con esputo.", "FR 28, SatO2
88% en aire ambiente, sibilancias difusas.", "EPOC exacerbado.", "Broncodilatador, corticoide,
oxígeno controlado y antibiótico."],
  ["2026-10-14T08:00", pr[2], "Disnea menor, tos productiva.", "SatO2 93% con oxígeno a 2 L/min,
sibilancias escasas.", "Mejoría parcial.", "Continuar manejo y retirar oxígeno según tolerancia."],
  ["2026-10-16T08:00", pr[2], "Sin disnea en reposo.", "SatO2 94% en aire ambiente, sin
sibilancias.", "Exacerbación resuelta.", "Educación en inhaladores y egreso al día siguiente."]]);
 sv(ho[4], [
  ["2026-10-12T07:00", pr[4], 110, 28, 37.2, 88, 148, 90], ["2026-10-12T15:00", pr[5], 104, 26, 37,
90, 144, 88],
```

```
    ["2026-10-13T08:00", pr[4], 96, 24, 36.9, 92, 140, 86], ["2026-10-14T08:00", pr[4], 90, 22, 36.8,
93, 136, 84],
    ["2026-10-15T08:00", pr[4], 84, 20, 36.7, 94, 132, 82], ["2026-10-16T08:00", pr[4], 80, 18, 36.6,
94, 128, 80]]);
  nt(ho[4], [
    ["2026-10-12T08:00", pr[4], "manana", "Se inicia oxígeno por cánula y nebulización, ansiedad
leve por la disnea."],
    ["2026-10-13T14:00", pr[5], "tarde", "Tolera dieta, expectoración mucosa moderada."],
    ["2026-10-15T21:00", pr[6], "noche", "Duerme sin oxígeno suplementario, sin disnea
nocturna."]]);
  // Hospitalización 6: cetoacidosis diabética (Valentina), con alta
  hs(ho[5], pa[5], "HOS-2026-0006", "alta",
    ad("2026-10-15T21:40", "urgencias", "Medicina interna", "M-09", "Poliuria, polidipsia y vómito de
24 horas", pr[3]),
    dx([["E11.1", "ingreso", "2026-10-15T22:10"], ["A09", "secundario", "2026-10-15T22:10"],
["E11.1", "principal", "2026-10-16T08:00"], ["E11.1", "egreso", "2026-10-20T10:00"]]),
    tx([[me[11], "0.1 UI/kg/h", "IV", "infusión continua", "2026-10-15T22:30", "2026-10-16T20:00",
pr[0]], [me[5], "20 UI", "SC", "cada 24 h", "2026-10-16T20:30", "2026-10-20T11:00", pr[0]], [me[6],
"20 mg", "oral", "cada 24 h", "2026-10-16T09:00", "2026-10-20T09:00", pr[0]]]),
    al("2026-10-20T11:00", "mejoria", "E11.1", "Control con endocrinología, insulina basal y
educación en glucometría", pr[0]));
  ev(ho[5], [
    ["2026-10-16T08:00", pr[0], "Náuseas, sed intensa, mejoría parcial del dolor abdominal.",
"Respiración profunda, mucosas secas, glucemia 410 mg/dL.", "Cetoacidosis diabética en
manejo.", "Infusión de insulina, líquidos IV y control de gases y electrolitos."],
    ["2026-10-17T08:00", pr[0], "Sin vómito, tolera líquidos.", "Glucemia 180 mg/dL, anión gap
normalizado.", "Cetoacidosis resuelta.", "Transición a insulina subcutánea."],
    ["2026-10-19T08:00", pr[0], "Asintomática, apetito conservado.", "Glucemias entre 110 y 160
mg/dL, signos vitales estables.", "Diabetes controlada con esquema basal.", "Educación
diabetológica y egreso programado."]]);
  sv(ho[5], [
    ["2026-10-15T22:30", pr[6], 118, 28, 37, 97, 98, 60], ["2026-10-16T02:00", pr[6], 110, 26, 36.9,
98, 102, 64],
    ["2026-10-16T08:00", pr[4], 100, 22, 36.8, 98, 108, 68], ["2026-10-17T08:00", pr[4], 88, 18, 36.7,
99, 112, 70],
    ["2026-10-18T08:00", pr[5], 82, 16, 36.6, 99, 114, 72], ["2026-10-19T08:00", pr[4], 78, 16, 36.6,
99, 116, 74]]);
  nt(ho[5], [
    ["2026-10-15T23:00", pr[6], "noche", "Se instala acceso venoso y líquidos IV, glucometrías
horarias."],
    ["2026-10-16T10:00", pr[4], "manana", "Persisten náuseas leves, se mantiene en ayuno y
monitoreo de glucemia."],
    ["2026-10-18T15:00", pr[5], "tarde", "Inicia dieta para diabético, tolera completa."]]);
  // Verificación de integridad referencial y de estado
  const hex = x => x.toHexString();
  const chk = { "Hospitalizaciones con paciente_id inexistente": [], "Tratamientos con
medicamento_id inexistente": [], "Diagnósticos con cie10_codigo ausente del catálogo": [],
"Registros hijos con hospitalizacion_id huérfano": [], "Estado inconsistente con objeto alta": [] };
```

```
  const kP = "Hospitalizaciones con paciente_id inexistente", kM = "Tratamientos con
medicamento_id inexistente", kD = "Diagnósticos con cie10_codigo ausente del catálogo", kH =
"Registros hijos con hospitalizacion_id huérfano", kE = "Estado inconsistente con objeto alta";
  const idsPac = new Set(db.pacientes.distinct("_id").map(hex));
  const idsMed = new Set(db.medicamentos.distinct("_id").map(hex));
  const idsHos = new Set(db.hospitalizaciones.distinct("_id").map(hex));
  const codigos = new Set(db.diagnosticos_cie10.distinct("_id"));
  db.hospitalizaciones.find().forEach(h => {
    if (!idsPac.has(hex(h.paciente_id))) chk[kP].push(h.numero_ingreso);
    (h.tratamientos || []).forEach(t => { if (!idsMed.has(hex(t.medicamento_id)))
chk[kM].push(h.numero_ingreso + " -> " + hex(t.medicamento_id)); });
    (h.diagnosticos || []).forEach(d => { if (!codigos.has(d.cie10_codigo))
chk[kD].push(h.numero_ingreso + " -> " + d.cie10_codigo); });
    if (h.estado === "activa" && h.alta) chk[kE].push(h.numero_ingreso + ": activa con alta");
    if (h.estado === "alta" && !h.alta) chk[kE].push(h.numero_ingreso + ": alta sin objeto alta");
  });
  ["signos_vitales", "notas_enfermeria", "evoluciones"].forEach(c =>
db.getCollection(c).find().forEach(x => { if (!idsHos.has(hex(x.hospitalizacion_id))) chk[kH].push(c +
" " + hex(x._id)); }));
  Object.keys(chk).forEach(k => print(k + ": " + (chk[k].length ? "\n  - " + chk[k].join("\n  - ") : "OK")));
  // Conteo de documentos por colección
  print("--- Conteo de documentos ---");
  ["pacientes", "profesionales", "medicamentos", "diagnosticos_cie10", "hospitalizaciones",
"signos_vitales", "notas_enfermeria", "evoluciones"].forEach(c => print(c + ": " +
db.getCollection(c).countDocuments()));
})();
```

La base `hce_hospitalizacion` se crea y se puebla con el script `scripts/creacion_y_carga.js`, que se pega completo en mongosh. El script se puede ejecutar varias veces: elimina las colecciones existentes antes de recrearlas.

Creación. El script crea las 8 colecciones con validadores `$jsonSchema` y sus índices (únicos, compuestos, multikey y de texto).

Carga. Inserta datos ficticios coherentes: 6 pacientes, 7 profesionales, 12 medicamentos, 10 códigos CIE-10 y 6 hospitalizaciones. Dos pacientes comparten el diagnóstico principal J18.9 y una hospitalización permanece activa (HOS-2026-0003). Cada hospitalización incluye admisión, diagnósticos, 2 o más tratamientos, 3 evoluciones SOAP, 6 registros de signos vitales y 3 notas de enfermería. Las hospitalizaciones con alta incluyen además el objeto `alta`.

Verificación de integridad. El script comprueba la existencia de paciente y medicamento referenciados, la presencia de cada código CIE-10 en el catálogo, la ausencia de registros

hijos huérfanos y la coherencia entre `estado` y el objeto `alta`. Cada comprobación imprime "OK" o la lista de problemas.

```
< Hospitalizaciones con paciente_id inexistente: OK
< Tratamientos con medicamento_id inexistente: OK
< Diagnósticos con cie10_codigo ausente del catálogo: OK
< Registros hijos con hospitalizacion_id huérfano: OK
< Estado inconsistente con objeto alta: OK
< --- Conteo de documentos ---
< pacientes: 6
< profesionales: 7
< medicamentos: 12
< diagnosticos_cie10: 10
< hospitalizaciones: 6
< signos_vitales: 36
< notas_enfermeria: 18
< evoluciones: 18
```

3. Consultas

Script: https://drive.google.com/file/d/1w7o3_UCEnOAbH3bi_-OLhdsQjn3LkNjs/view?usp=drive_link

```
// hce_consultas.js
(function () {
  db = db.getSiblingDB("hce_hospitalizacion");
  // Consulta 1: pacientes por diagnóstico
  print("\n=== 1. Pacientes por diagnóstico | Objetivo: listar quién tiene cada CIE-10 | Valida: $unwind de array embebido y referencias a catálogo y pacientes ===");
  db.hospitalizaciones.aggregate([
    { unwind: "diagnosticos" },
    { $match: { "diagnosticos.tipo": "principal" } },
```

```
  { $lookup: { from: "diagnosticos_cie10", localField: "diagnosticos.cie10_codigo", foreignField: "_id", as: "dx" } },
  { $lookup: { from: "pacientes", localField: "paciente_id", foreignField: "_id", as: "pac" } },
  { $unwind: "dx" },
  { $unwind: "pac" },
  { $group: { _id: { codigo: "$dx._id", descripcion: "$dx.descripcion" }, pacientes: { $addToSet: { nombre: { $concat: ["pac.nombres", " ", "$pac.apellidos"] }, documento: { $concat: ["pac.tipo_documento", " ", "$pac.numero_documento"] } } } } },
  { $project: { _id: 0, codigo: "$_id.codigo", descripcion: "$_id.descripcion", pacientes: 1 } },
  { $sort: { codigo: 1 } }
 ]).forEach(printjson);
 // Consulta 2: evolución clínica cronológica de la hospitalización del primer paciente
 print("\n=== 2. Evolución SOAP cronológica | Objetivo: línea de tiempo médica del primer paciente | Valida: colección referenciada evoluciones, índice (hospitalizacion_id, fecha) y catálogo profesionales ===");
 const p1 = db.pacientes.find().sort({ numero_documento: 1 }).limit(1).toArray()[0];
 const h1 = db.hospitalizaciones.findOne({ paciente_id: p1._id });
 print("Hospitalización: " + h1.numero_ingreso);
 db.evoluciones.aggregate([
  { $match: { hospitalizacion_id: h1._id } },
  { $sort: { fecha: 1 } },
  { $lookup: { from: "profesionales", localField: "profesional_id", foreignField: "_id", as: "prof" } },
  { $unwind: "prof" },
  { $project: { _id: 0, fecha: 1, profesional: { $concat: ["prof.nombres", " ", "$prof.apellidos"] }, subjetivo: 1, objetivo: 1, analisis: 1, plan: 1 } }
 ]).forEach(printjson);
 // Consulta 3: medicamentos asignados a un paciente
 print("\n=== 3. Medicamentos de un paciente | Objetivo: tratamientos con datos del catálogo | Valida: tratamientos embebidos con medicamento_id referenciado ===");
 print("Paciente: " + p1.nombres + " " + p1.apellidos);
 db.hospitalizaciones.aggregate([
  { $match: { paciente_id: p1._id } },
  { $unwind: "tratamientos" },
```

```
  { $lookup: { from: "medicamentos", localField: "tratamientos.medicamento_id", foreignField: "_id", as: "med" } },
  { unwind: "med" },
  { $project: { _id: 0, ingreso: "$numero_ingreso", medicamento: "med.nombre_generico", dosis: "tratamientos.dosis", via: "tratamientos.via", frecuencia: "tratamientos.frecuencia", fecha_inicio: "tratamientos.fecha_inicio", fecha_fin: "tratamientos.fecha_fin" } },
  { $sort: { fecha_inicio: 1 } }
 ]).forEach(printjson);
 // Consulta 4: hospitalizaciones activas con último signo vital
 print("\n=== 4. Activas con último signo vital | Objetivo: estado actual de pacientes hospitalizados | Valida: $lookup con sub-pipeline sobre colección referenciada e índice (hospitalizacion_id, fecha desc) ===");
 db.hospitalizaciones.aggregate([
  { $match: { estado: "activa" } },
  { $lookup: { from: "signos_vitales", let: { hid: "$_id" }, pipeline: [
    { $match: { $expr: { eq: ["hospitalizacion_id", "$$hid"] } } },
    { $sort: { fecha: -1 } },
    { $limit: 1 },
    { $project: { _id: 0, fecha: 1, fc: 1, fr: 1, temp: 1, sat_o2: 1, ta_sis: 1, ta_dia: 1 } }
  ], as: "ultimo_signo_vital" } },
  { $unwind: { path: "$ultimo_signo_vital", preserveNullAndEmptyArrays: true } },
  { $project: { _id: 0, numero_ingreso: 1, estado: 1, servicio: "$admision.servicio", cama: "$admision.cama", ultimo_signo_vital: 1 } }
 ]).forEach(printjson);
 // Consulta 5: explain de búsqueda por código CIE-10
 print("\n=== 5. Explain por diagnosticos.cie10_codigo | Objetivo: comprobar uso de índice | Valida: índice multikey sobre array embebido de diagnósticos ===");
 const ex = db.hospitalizaciones.find({ "diagnosticos.cie10_codigo": "J18.9" }).explain("executionStats");
 const plan = ex.queryPlanner.winningPlan;
 const nodo = plan.inputStage || (plan.queryPlan && plan.queryPlan.inputStage) || plan;
 const raiz = plan.queryPlan || plan;
 printjson({
```

```
  "winningPlan.stage": raiz.stage,
  indexName: nodo.indexName || (nodo.inputStage && nodo.inputStage.indexName),
  totalDocsExamined: ex.executionStats.totalDocsExamined,
  nReturned: ex.executionStats.nReturned
 });
 // Consulta 6: vista de auditoría sin identidad
 print("\n=== 6. Auditoría sin identidad | Objetivo: datos clínicos anonimizados | Valida: separación de privacidad entre hospitalizaciones y pacientes ===");
 db.hospitalizaciones.aggregate([
  { $project: { _id: 0, numero_ingreso: 1, estado: 1, servicio: "$admision.servicio", fecha_admision: "admision.fecha", fecha_alta: "alta.fecha", diagnosticos: 1 } },
  { $sort: { numero_ingreso: 1 } }
 ]).forEach(printjson);
})();
```

Consulta 1. Pacientes por diagnóstico.

Objetivo: listar, por código CIE-10 principal, quiénes lo tienen.

Valida: $unwind sobre array embebido y $lookup a catálogo y pacientes.

Verificar: J18.9 debe agrupar a Juan Camilo Pérez Gómez y María Fernanda Gómez Ruiz.

Resultado:

```
{
  pacientes: [
    { nombre: 'María Fernanda Gómez Ruiz', documento: 'CC 1000000002' },
    { nombre: 'Juan Camilo Pérez Gómez', documento: 'CC 1000000001' }
  ],
  codigo: 'J18.9',
  descripcion: 'Neumonía, no especificada'
}
```

```
=== 1. Pacientes por diagnóstico | Objetivo: listar quién tiene cada CIE-10 | Valida: $unwind de array embebido y referencias a catálogo y pacientes ===
[
  {
    pacientes: [ { nombre: 'Valentina Ríos Arango', documento: 'CC 1000000006' } ],
    codigo: 'E11.1',
    descripcion: 'Diabetes mellitus tipo 2 con cetoacidosis'
  }
[
  {
    pacientes: [ { nombre: 'Luis Alberto Torres Mora', documento: 'CC 1000000003' } ],
    codigo: 'I50.9',
    descripcion: 'Insuficiencia cardíaca, no especificada'
  }
[
  {
    pacientes: [
      { nombre: 'María Fernanda Gómez Ruiz', documento: 'CC 1000000002' },
      { nombre: 'Juan Camilo Pérez Gómez', documento: 'CC 1000000001' }
    ],
    codigo: 'J18.9',
    descripcion: 'Neumonía, no especificada'
  }
[
  {
    pacientes: [ { nombre: 'Pedro Pablo Salazar Henao', documento: 'CC 1000000005' } ],
    codigo: 'J44.1',
    descripcion: 'Enfermedad pulmonar obstructiva crónica con exacerbación aguda'
  }
[
  {
    pacientes: [ { nombre: 'Ana Sofía Cardona Londoño', documento: 'CC 1000000004' } ],
    codigo: 'N39.0',
    descripcion: 'Infección de vías urinarias, sitio no especificado'
  }
```

Consulta 2. Evolución SOAP cronológica de HOS-2026-0001.

Objetivo: línea de tiempo médica del primer paciente, con nombre del profesional.

Valida: colección referenciada evoluciones, índice (hospitalizacion_id, fecha) y catálogo profesionales.

Resultado:

{

fecha: 2026-10-01T16:00:00.000Z,

subjetivo: 'Tos con expectoración, fiebre y malestar general.',

objetivo: 'T 38.4 °C, FR 22, estertores en base derecha, SatO2 93%.',

analisis: 'Neumonía adquirida en comunidad moderada.',

plan: 'Ceftriaxona IV, azitromicina y oxígeno si hay desaturación.',

profesional: 'Laura Restrepo Vélez'

}

```
=== 2. Evolución SOAP cronológica | Objetivo: línea de tiempo médica del primer paciente | Valida: colección referenciada evoluciones, índice (hospitalizacion_id, fecha) y catálogo profesionales ===
Hospitalización: HOS-2026-0001
{
  fecha: 2026-10-01T16:00:00.000Z,
  subjetivo: 'Tos con expectoración, fiebre y malestar general.',
  objetivo: 'T 38.4 °C, FR 22, estertores en base derecha, SatO2 93%.',
  analisis: 'Neumonía adquirida en comunidad moderada.',
  plan: 'Ceftriaxona IV, azitromicina y oxígeno si hay desaturación.',
  profesional: 'Laura Restrepo Vélez'
}
{
  fecha: 2026-10-03T08:00:00.000Z,
  subjetivo: 'Menos tos, sin disnea.',
  objetivo: 'Afebril 24 h, SatO2 95% en aire ambiente.',
  analisis: 'Respuesta clínica favorable.',
  plan: 'Continuar antibiótico y vigilar signos vitales.',
  profesional: 'Laura Restrepo Vélez'
}
{
  fecha: 2026-10-05T08:00:00.000Z,
  subjetivo: 'Asintomático, tolera vía oral.',
  objetivo: 'Afebril, SatO2 96%, murmullo vesicular conservado.',
  analisis: 'Neumonía en resolución.',
  plan: 'Programar egreso al completar el esquema.',
  profesional: 'Laura Restrepo Vélez'
}
```

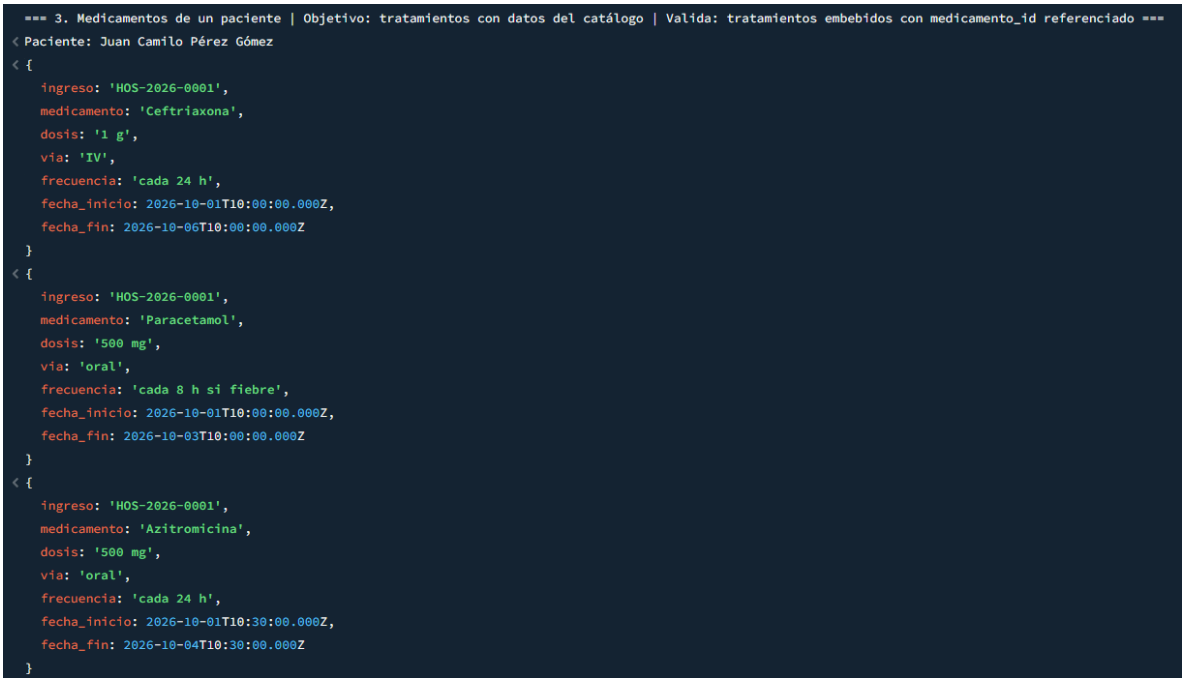
Consulta 3. Medicamentos de un paciente.

Objetivo: tratamientos con nombre genérico, dosis, vía, frecuencia y fechas.

Valida: tratamientos embebidos con medicamento_id referenciado.

Resultado:

{
  ingreso: 'HOS-2026-0001',
  medicamento: 'Ceftriaxona',
  dosis: '1 g',
  via: 'IV',
  frecuencia: 'cada 24 h',
  fecha_inicio: 2026-10-01T10:00:00.000Z,
  fecha_fin: 2026-10-06T10:00:00.000Z
}



Consulta 4. Hospitalizaciones activas con último signo vital.

Objetivo: estado actual de los pacientes hospitalizados.

Valida: $lookup con sub-pipeline (sort descendente y limit 1) sobre colección referenciada.

Resultado:

{

numero_ingreso: 'HOS-2026-0003',

estado: 'activa',

ultimo_signo_vital: {

fecha: 2026-10-25T08:00:00.000Z,

fc: 80,

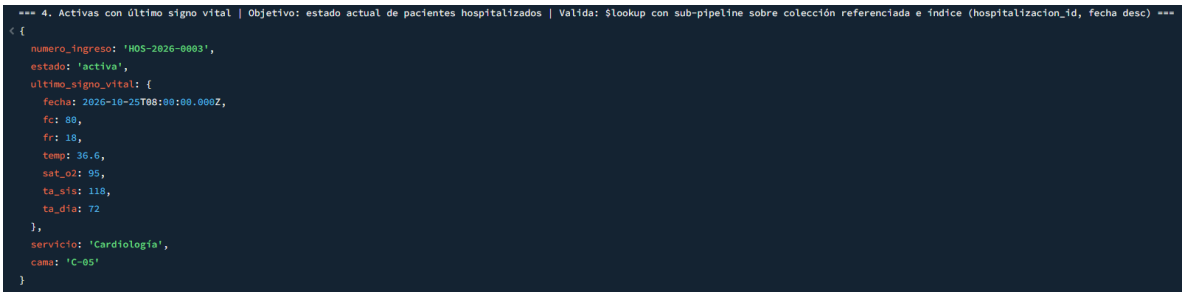
fr: 18,

temp: 36.6,

sat_o2: 95,

ta_sis: 118,

```
    ta_dia: 72
  },
  servicio: 'Cardiología',
  cama: 'C-05'
}
```



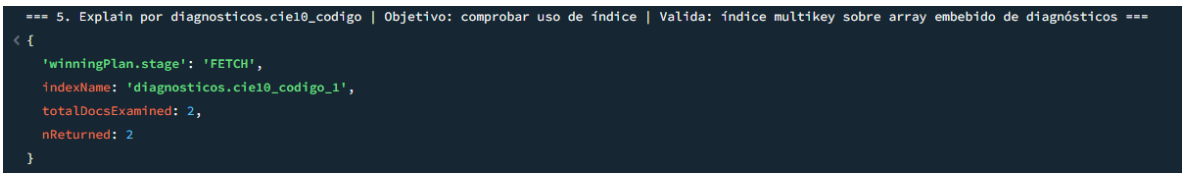
Consulta 5. `explain("executionStats")` por `diagnosticos.cie10_codigo`.

Objetivo: comprobar el uso de índice.

Valida: índice multikey sobre el array embebido de diagnósticos.

Resultado (winningPlan.stage, indexName, totalDocsExamined, nReturned):

```
{
  'winningPlan.stage': 'FETCH',
  indexName: 'diagnosticos.cie10_codigo_1',
  totalDocsExamined: 2,
  nReturned: 2
}
```



Consulta 6. Vista de auditoría sin identidad.

Objetivo: exponer datos clínicos sin paciente_id ni datos de pacientes.

Valida: separación de privacidad entre colecciones.

Resultado:


```
{
  numero_ingreso: 'HOS-2026-0001',
  estado: 'alta',
  diagnosticos: [
    {
      cie10_codigo: 'J18.9',
      tipo: 'ingreso',
      fecha: 2026-10-01T09:00:00.000Z
    },
    {
      cie10_codigo: 'R50.9',
      tipo: 'secundario',
      fecha: 2026-10-01T09:00:00.000Z
    },
    {
      cie10_codigo: 'J18.9',
      tipo: 'principal',
      fecha: 2026-10-02T08:00:00.000Z
    },
    {
      cie10_codigo: 'J18.9',
      tipo: 'egreso',
      fecha: 2026-10-06T10:00:00.000Z
    }
  ],
  servicio: 'Medicina interna',
  fecha_admision: 2026-10-01T08:30:00.000Z,
  fecha_alta: 2026-10-06T11:00:00.000Z
}
```

```
=== 6. Auditoría sin identidad | Objetivo: datos clínicos anonimizados | Valida: separación de privacidad entre hospitalizaciones y pacientes ===
{
  numero_ingreso: 'HOS-2026-0001',
  estado: 'alta',
  diagnosticos: [
    {
      cie10_codigo: 'J18.9',
      tipo: 'ingreso',
      fecha: 2026-10-01T09:00:00.000Z
    },
    {
      cie10_codigo: 'R50.9',
      tipo: 'secundario',
      fecha: 2026-10-01T09:00:00.000Z
    },
    {
      cie10_codigo: 'J18.9',
      tipo: 'principal',
      fecha: 2026-10-02T08:00:00.000Z
    },
    {
      cie10_codigo: 'J18.9',
      tipo: 'egreso',
      fecha: 2026-10-06T10:00:00.000Z
    }
  ],
  servicio: 'Medicina interna',
  fecha_admision: 2026-10-01T08:30:00.000Z,
  fecha_alta: 2026-10-06T11:00:00.000Z
}
{
  numero_ingreso: 'HOS-2026-0002',
  estado: 'alta',
  diagnosticos: [
    {
      cie10_codigo: 'J18.9',
      tipo: 'ingreso',
      fecha: 2026-10-03T14:40:00.000Z
```

4. Evidencia de IA

Herramienta	Prompt (resumen)	Aporte	Corrección manual
Claude	Diseñar el modelo híbrido con tablas, JSON de ejemplo, $jsonSchema, índices, Mermaid y comparación embebido vs referenciado	Propuesta de esquema, validadores, índices y diagrama	Se revisó críticamente y se corrigieron tres puntos: (1) el script no se podía ejecutar dos veces porque `createCollection` falla si la colección existe; (2) MongoDB no valida las referencias (`medicamento_id`, `cie10_codigo`, `paciente_id`), lo que obligó a agregar verificación de integridad; (3) la IA presentaba los datos clínicos como "sin identidad", pero `paciente_id` solo seudonimiza, así que se reformuló como seudonimización y no anonimización.
Claude	Script único de creación y carga con datos ficticios y verificación de integridad	creacion_y_carga.js	Se pidió que fuera un solo bloque para pegar en mongosh (con `drop` previo para repetir la ejecución, `getSiblingDB` en vez de `use`, y función autoejecutable). Se añadió el chequeo

			de huérfanos y de consistencia `estado` vs `alta`.
Claude	6 consultas de validación con `$lookup`, sub-pipeline y `explain`	`consultas.js`	Se unificó en un solo script ejecutable. Se verificaron los resultados contra los datos cargados y se confirmó con `explain` que la consulta por diagnóstico usa índice (IXSCAN) y no recorre toda la colección.
Claude	Estructura del repositorio, README, guía de capturas	Repositorio base	Se ajustó a los dos scripts reales (`creacion_y_carga.js` y `consultas.js`). Se eliminó Docker y mongo-express del README porque toda la ejecución y las capturas se hicieron manualmente en MongoDB Compass.

Historial de conversación IA

https://claude.ai/share/4c8a26cd-19fe-4ddc-aa0e-0e5a25e87a35

La IA propuso el esquema inicial y el equipo lo revisó críticamente antes de adoptarlo. Correcciones realizadas:

- **Script no re-ejecutable:** la primera versión fallaba al correrse dos veces. Se agregó el `drop` previo de colecciones y se envolvió todo en una función autoejecutable para evitar constantes globales duplicadas.

Falta de validación de referencias: $jsonSchema valida estructura y tipos, no existencia del documento referenciado. Se añadió la verificación de integridad referencial al script.

Sobrestimación de "sin identidad": se había presentado la vista de auditoría como anónima. Solo está seudonimizada y contiene texto libre potencialmente identificable (ver sección 6).

5. Reflexión ética

Los datos de salud son datos sensibles según la Ley 1581 de 2012 (protección de datos personales en Colombia), que exige autorización expresa y reglas reforzadas de tratamiento. La Resolución 1995 de 1999 regula la historia clínica y establece su carácter reservado, la integridad del registro y su conservación por 20 años. Las decisiones de modelado tienen efectos directos sobre ambos marcos.

a) Privacidad.

Separar pacientes de hospitalizaciones solo seudonimiza: paciente_id permite reidentificar a quien tenga acceso a ambas colecciones. No es anonimización. Además, motivo_consulta, las notas de enfermería y las evoluciones son texto libre y pueden contener nombres, lugares u otros datos identificables, de modo que incluso una vista sin paciente_id puede revelar identidad por contexto. La separación reduce el riesgo, pero no lo elimina.

b) Seguridad.

La separación permite control de acceso por colección: el personal de auditoría o estadística accede a datos clínicos sin permisos sobre pacientes, mediante roles de mínimo privilegio. Las vistas sin identidad sirven para auditoría, siempre sobre la base de que son seudonimizadas, no anónimas, y requieren cifrado en tránsito y en reposo y registro de accesos.

c) Consistencia.

MongoDB no impone integridad referencial: un paciente_id o medicamento_id puede apuntar a un documento inexistente sin error. Las mitigaciones son el script de verificación (que detecta huérfanos), las transacciones multi-documento cuando una operación toca varias colecciones y la validación en la aplicación antes de escribir. Ninguna sustituye una restricción declarativa del motor.

d) Embebido vs referenciado.

Embeber da atomicidad por documento: la admisión, los diagnósticos y el alta se escriben de forma consistente en una sola operación. A cambio, los arrays sin tope degradan el rendimiento y amenazan el límite de 16 MB, riesgo inaceptable en una estancia larga. Por eso lo que crece se referencia, aceptando que registrar una evolución junto con un cambio de tratamiento ya no es atómico sin transacción.

e) Trazabilidad y auditoría.

La Resolución 1995 exige que la historia clínica sea íntegra y no se altere sin rastro. El modelo actual no registra quién modificó qué ni cuándo. Haría falta añadir campos de auditoría (usuario, fecha de creación y modificación), conservar versiones en lugar de sobrescribir y registrar los accesos de lectura a datos sensibles.

En conjunto, el diseño técnico reduce riesgos, pero el cumplimiento depende de controles organizacionales: autorización del titular, política de acceso y conservación documentada.

6. División del trabajo

Integrante	Rol	Tareas
Santiago Padierna	Diseño del modelo	Esquema, justificación embebido vs referenciado, diagrama
Santiago Padierna	Implementación	Docker, creación de colecciones, validadores e índices
Santiago Padierna	Carga y verificación	Datos ficticios, verificación de integridad, conteos
Santiago Padierna	Consultas	Las 6 consultas y análisis de explain
Santiago Padierna	Documentación	README, repositorio GitHub, capturas
Santiago Padierna	Ética e IA	Reflexión ética y evidencia de uso de IA